



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.XX.XX Название направления

О Т Ч Е Т
по лабораторной работе № 5

Название:

Дисциплина: Языки интернет-программирования

Студент

ИУ6-33Б

(Группа)

(Подпись, дата)

Б.П.Проценко

(И.О. Фамилия)

Преподавател

ь

(Подпись, дата)

И.О. Фамилия

(И.О. Фамилия)

Москва, 2024

Задание 1) Вам необходимо написать функцию `calculator` следующего вида:

```
func calculator(firstChan <-chan int, secondChan <-chan int, stopChan <-chan struct{}) <-chan int
```

Функция получает в качестве аргументов 3 канала, и возвращает канал типа `<-chan int`.

- в случае, если аргумент будет получен из канала `firstChan`, в выходной (возвращенный) канал вы должны отправить квадрат аргумента.
- в случае, если аргумент будет получен из канала `secondChan`, в выходной (возвращенный) канал вы должны отправить результат умножения аргумента на 3.
- в случае, если аргумент будет получен из канала `stopChan`, нужно просто завершить работу функции.

Функция `calculator` должна быть неблокирующей, сразу возвращая управление. Ваша функция получит всего одно значение в один из каналов - получили значение, обработали его, завершили работу.

После завершения работы необходимо освободить ресурсы, закрыв выходной канал, если вы этого не сделаете, то превысите предельное время работы.

Решение:

```
func calculator(firstChan <-chan int, secondChan <-chan int, stopChan <-chan struct{})
<-chan int {

    resultChan := make(chan int)

    go func() {
        defer close(resultChan)

        select {
        case val := <-firstChan:
            resultChan <- val * val
        case val := <-secondChan:
            resultChan <- val * 3
        case <-stopChan:
            return
        }
    }()

    return resultChan
}
```

Задание 2) Напишите элемент конвейера (функцию), что запоминает предыдущее значение и отправляет значения на следующий этап конвейера только если оно отличается от того, что пришло ранее. Ваша функция должна принимать два канала - `inputStream` и `outputStream`, в первый вы будете получать строки, во второй вы должны отправлять значения без повторов. В итоге в `outputStream` должны остаться значения, которые не повторяются подряд. Не забудьте закрыть канал ;)

Функция **должна** называться `removeDuplicates()`

Выводить или вводить ничего не нужно!

Решение:

```
func removeDuplicates(inputStream <-chan string, outputStream chan<- string) {
    defer close(outputStream)

    var msg string
    first:= true

    for val := range inputStream {
        if first|| val != msg {
            outputStream <- val
            msg = val
            first= false
        }
    }
}
```

Задание 3)Внутри функции main (функцию объявлять не нужно), вам необходимо в отдельных горутинах вызвать функцию work() 10 раз и дождаться результатов выполнения вызванных функций.

Функция work() ничего не принимает и не возвращает. Пакет "sync" уже импортирован.

```
var wg sync.WaitGroup
```

```
for i := 0; i < 10; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        work()
    }()
}
```

```
wg.Wait()
```